StudyCompass

Context-Aware Study Resource Recommendation API

Personalized Learning Resource Recommendation System

Presented by: Rajavarshini C
Register Number: 23MIC0142
Partner Name: Durgadevi M (23MIC0139)
Course: Advanced Python Programming

Introduction

- StudyCompass is a Python-based personalized learning resource recommendation system.
- It recommends educational resources based on the student's topic, difficulty level, previous assessment score, learning goal, and preferred learning format.
- The system provides a Streamlit interface and a FastAPI endpoint for accessing recommendations.

Problem Statement

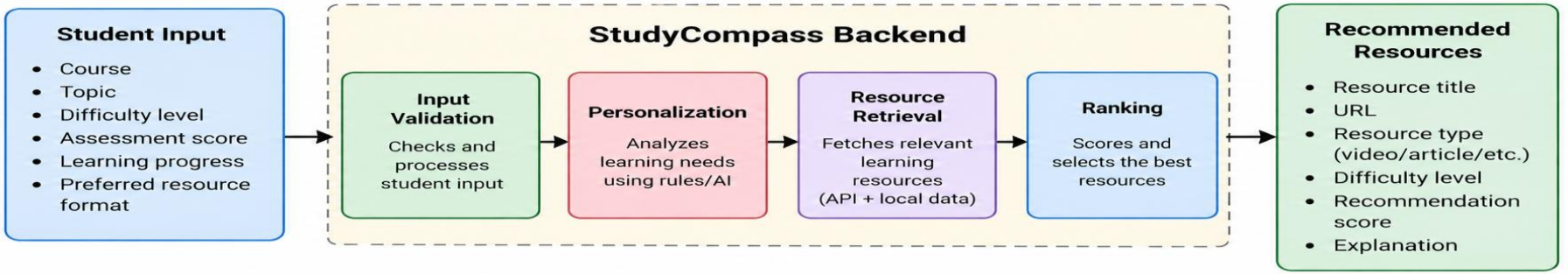
- Students often spend time searching for learning resources that match their current knowledge and learning needs.
- Generic resources may not suit every student's learning level or preferred format.
- StudyCompass addresses this problem by ranking learning resources according to topic relevance, difficulty, format, performance, and resource quality.

Objectives

- Generate personalized learning resource recommendations.
- Consider student performance and learning goals.
- Support multiple learning resource formats.
- Rank resources using weighted scoring.
- Provide explanations for recommendations.
- Expose recommendation functionality through a REST API.
- Support integration with external educational platforms.

System Overview

- StudyCompass collects information about the student's learning requirements.
- The system validates the input, interprets the learning context, retrieves resources, and ranks them using a weighted scoring mechanism.
- The final recommendations are displayed through the Streamlit interface or returned through the FastAPI endpoint.



Technologies Used

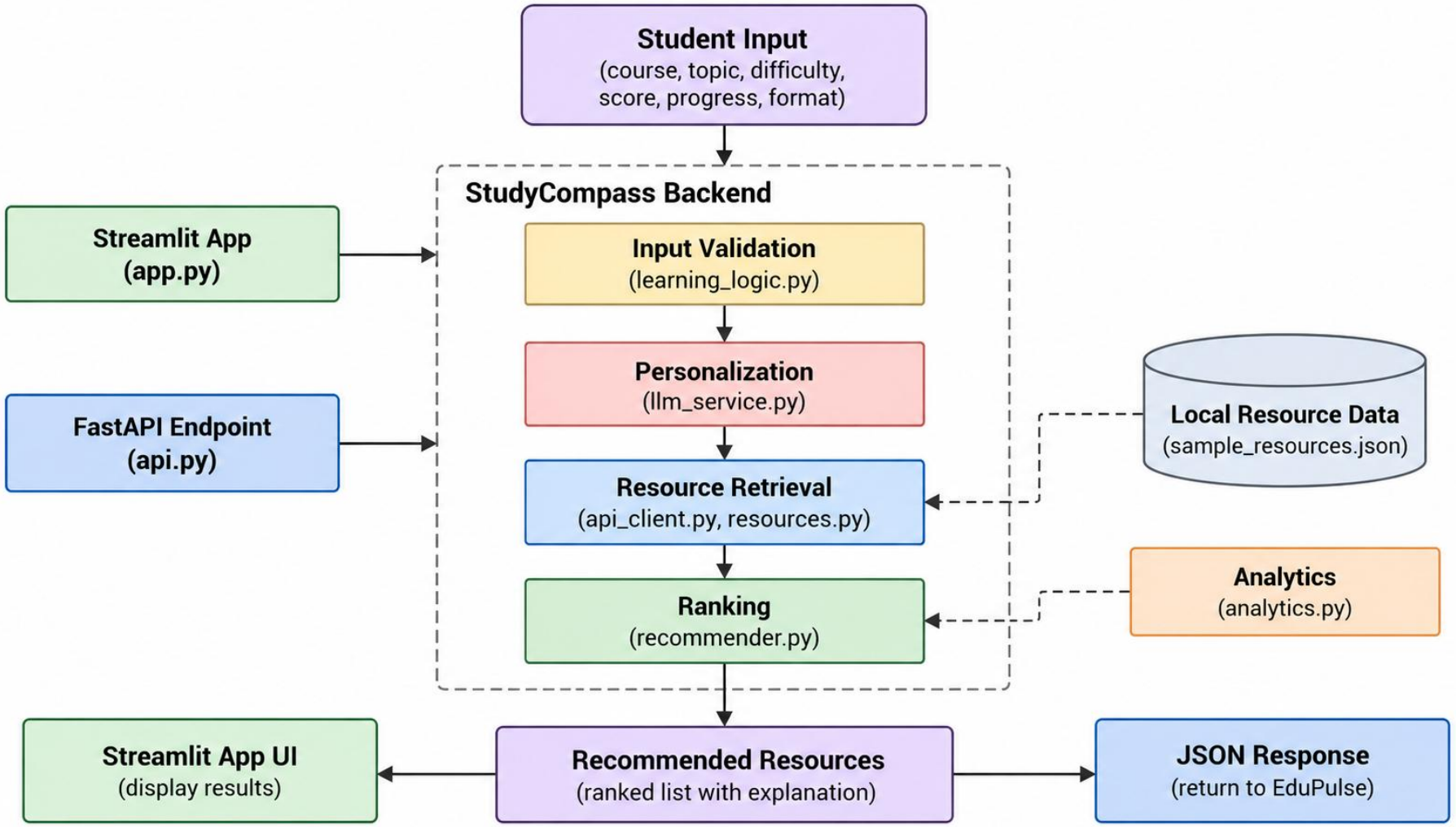
Technology	Purpose
Python	Core application logic
Streamlit	User interface
FastAPI	REST API development
Gemini API	AI-assisted personalization
DEV Community API	Retrieval of live article resources
JSON	Local resource storage and data exchange
Uvicorn	Running the API server
Pytest	Testing the application

System Architecture

The system has two entry points:

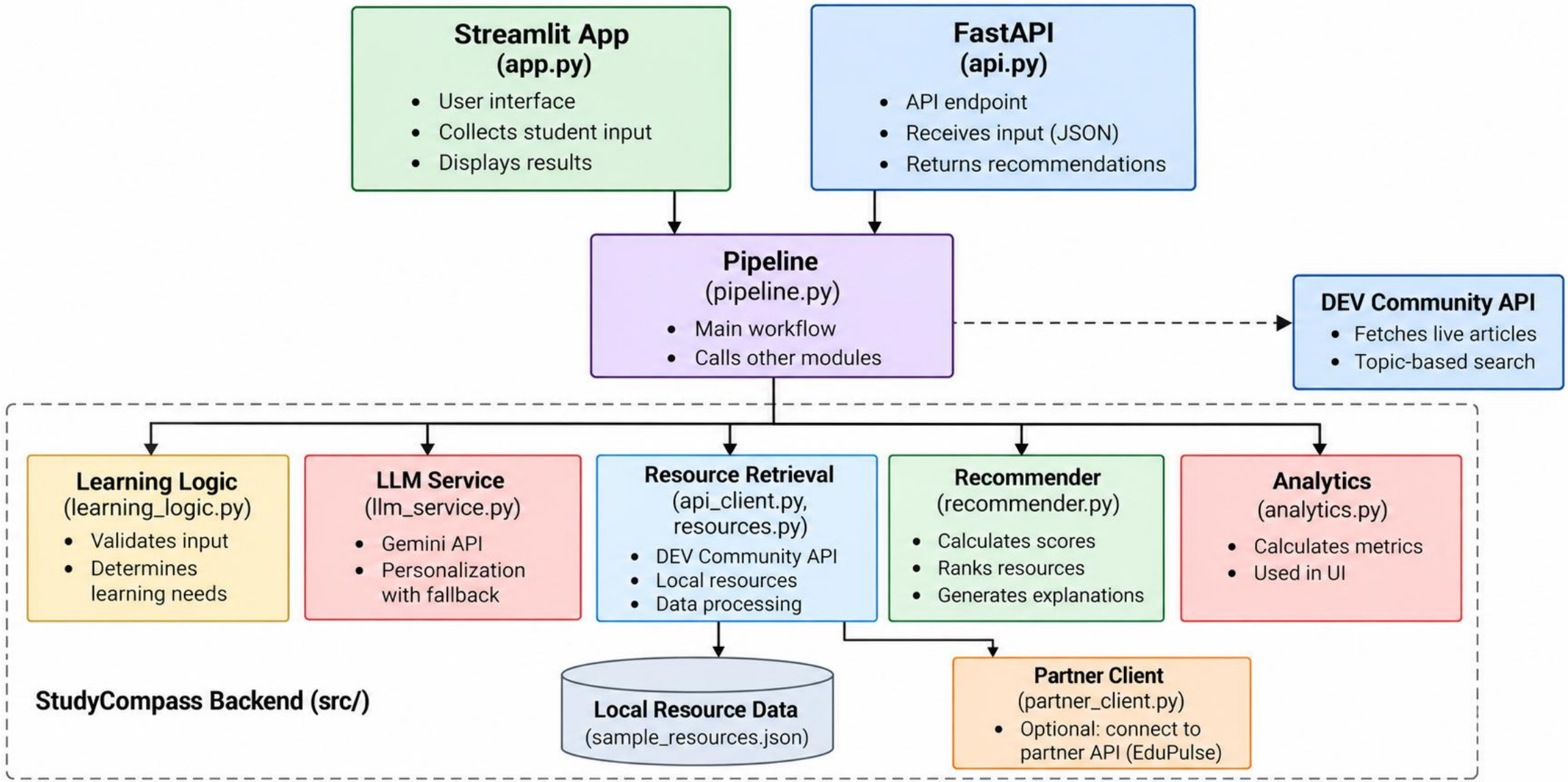
- **1. Streamlit Application:** Collects student information and displays recommendations through a web interface.
- **2. FastAPI Backend:** Accepts HTTP requests and returns recommendation results in JSON format.
- Both entry points use the same recommendation pipeline, which validates inputs, personalizes learning context, retrieves resources, and ranks the results.

Architecture Diagram



Python File Structure and Flow

File	Responsibility
app.py	Streamlit interface and user interaction
api.py	FastAPI endpoints for recommendation requests
src/pipeline.py	Coordinates the complete recommendation process
src/learning_logic.py	Validates student input and determines learning needs
src/llm_service.py	Gemini-based personalization with rule-based fallback
src/api_client.py	Retrieves live articles from the DEV Community API
src/resources.py	Manages resource data, validation, and duplicates
src/recommender.py	Calculates recommendation scores and ranks resources
src/partner_client.py	Optional communication with a partner application
src/analytics.py	Calculates analytics and metrics for the interface
src/config.py	Reads application configuration and environment settings
data/sample_resources.json	Stores curated learning resources


Streamlit App
(app.py)
User interface
Collects student input
Displays results
FastAPI
(api.py)
API endpoint
Receives input (JSON)
Returns recommendations
Pipeline
(pipeline.py)
Main workflow
Calls other modules
DEV Community API
Fetches live articles
Topic-based search
Learning Logic
(learning_logic.py)
Validates input
Determines learning needs
LLM Service
(llm_service.py)
Gemini API
Personalization with fallback
Resource Retrieval
(api_client.py, resources.py)
DEV Community API
Local resources
Data processing
Recommender
(recommender.py)
Calculates scores
Ranks resources
Generates explanations
Analytics
(analytics.py)
Calculates metrics
Used in UI
StudyCompass Backend (src/)
Local Resource Data
(sample_resources.json)
Partner Client
(partner_client.py)
Optional: connect to partner API (EduPulse)

Recommendation Algorithm

StudyCompass uses a weighted scoring algorithm to rank learning resources.

The final score is calculated using five factors:

- Topic relevance: 30%
- Difficulty match: 20%
- Preferred format match: 20%
- Performance suitability: 20%
- Resource quality: 10%

The resources are ranked according to their final scores, and explanations are generated to help users understand the recommendations.

API Implementation

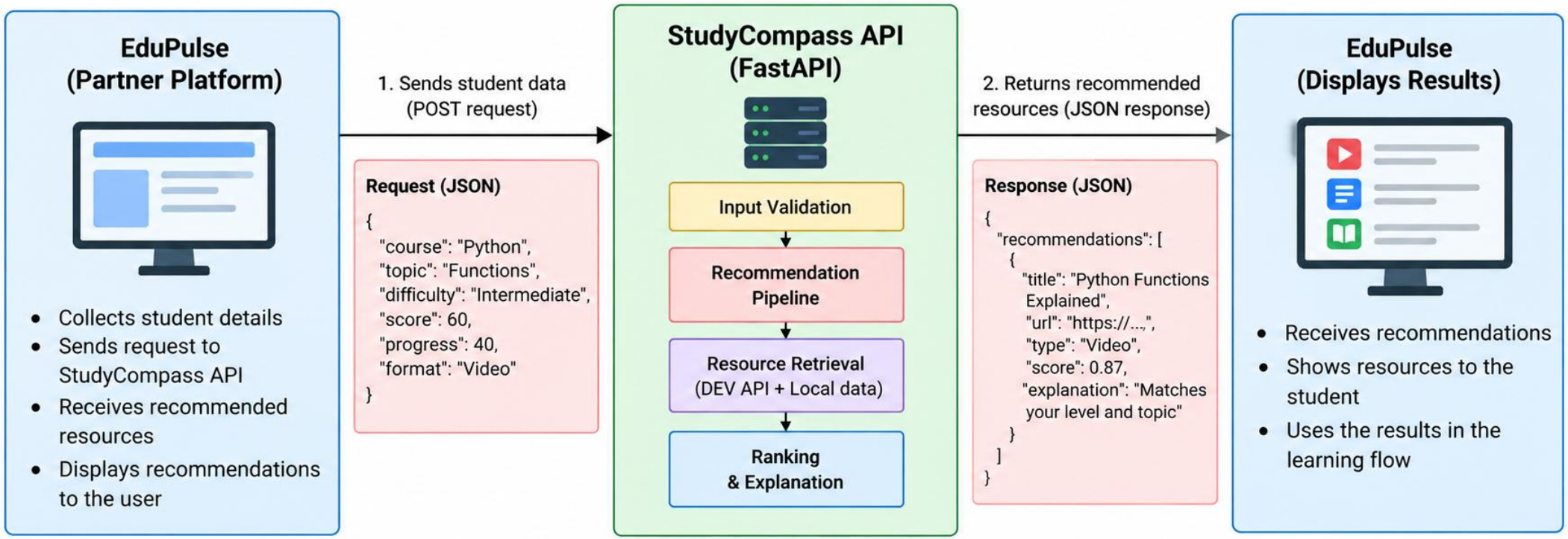
- StudyCompass provides a REST API using FastAPI.
- **Endpoint:** POST /api/recommendations
- The API accepts student learning information, including the course, topic, difficulty, assessment score, learning progress, and preferred resource type.
- It processes the request through the shared recommendation pipeline and returns recommended resources in JSON format.
- The API also validates incoming data and reports invalid requests.

Output and Results

- The system generates personalized recommendations based on the student's learning requirements.
- The results include:
- Resource title and URL.
- Resource type and source.
- Recommendation score.
- Explanation of the recommendation.
- Relevant resource metadata.
- The interface also includes analytics features for exploring recommendation results.

Integration with EduPulse

- StudyCompass is designed to provide recommendation functionality to external educational applications.
- EduPulse can send student learning information to the StudyCompass API and receive recommended learning resources.
- The optional partner client also supports configurable communication with an external recommendation service.
- The integration workflow depends on the API configuration and compatibility of the request and response formats.


EduPulse
(Partner Platform)
Collects student details
Sends request to StudyCompass API
Receives recommended resources
Displays recommendations to the user
1. Sends student data
(POST request)
Request (JSON)
{
"course": "Python",
"topic": "Functions",
"difficulty": "Intermediate",
"score": 60,
"progress": 40,
"format": "Video"
}
StudyCompass API
(FastAPI)
Input Validation
Recommendation Pipeline
Resource Retrieval
(DEV API + Local data)
Ranking
& Explanation
2. Returns recommended resources (JSON response)
Response (JSON)
{
"recommendations": [
{
"title": "Python Functions Explained",
"url": "https://...",
"type": "Video",
"score": 0.87,
"explanation": "Matches your level and topic"
}
]
}
EduPulse
(Displays Results)
Receives recommendations
Shows resources to the student
Uses the results in the learning flow

Conclusion

- StudyCompass demonstrates personalized learning resource recommendation using Python.
- It combines student-context validation, AI-assisted personalization, resource retrieval, and weighted scoring.
- The modular architecture supports both a Streamlit interface and a FastAPI service.
- The project provides a foundation for integrating personalized recommendations into educational platforms.

Future Enhancements

- Deploy the API for external access.
- Improve recommendation personalization.
- Expand support for additional resource sources.
- Enhance recommendation analytics.
- Complete and validate external platform integration.

Thank You